

1 Úvod

1.1 Cíl projektu

Cílem této semestrální práce je návrh a realizace komplexní testovací strategie pro herní engine **DaoEngine**. Práce se zaměřuje na verifikaci klíčových mechanik RPG systému, stabilitu procedurálního generování světa a správnost výpočtů v bojovém subsystému. Hlavním výstupem je prokázání schopnosti aplikovat pokročilé metodiky testování softwaru na netriviální aplikaci v jazyce Java.

1.2 Cíl dokumentu

Tento report slouží jako ucelená dokumentace procesu testování. Obsahuje analýzu architektury z pohledu testera, identifikaci kritických rizik, návrh testovacích scénářů a vyhodnocení dosaženého pokrytí (MCC, MC/DC, BVA). Dokumentace mapuje cestu od definice SUT (System Under Test) až po finální integrační verifikaci celého herního cyklu.

1.3 Autor

Autorem celého projektu DaoEngine i této testovací dokumentace je Charles (já), který aplikaci navrhl jako samostatný projekt. Vzhledem k tomu, že engine obsahuje komplexní matematické modely pro RPG atributy a asynchronní operace, byla kladen velký důraz na automatizaci testů, aby byla zajištěna dlouhodobá stabilita kódu.

1.4 Výběr aplikace (SUT)

Pro verifikaci byl zvolen **DaoEngine** (moje semestrální práce z předmětu JAVA) 2D RPG engine postavený na frameworku JavaFX, který implementuje specifické mechaniky žánru "cultivation" (xianxia). Volba padla na tento projekt především kvůli jeho vysoké vnitřní komplexitě a množství stavových proměnných, které představují ideální objekt pro aplikaci metodik jako jsou třídy ekvivalence nebo stavové testování.

2 Popis testované aplikace (SUT)

2.1 Funkční specifikace a architektura

DaoEngine není pouze statickou hrou, ale modulárním systémem, který orchestruje několik kritických subsystémů. Z pohledu testování je aplikace rozdělena na logické celky, u nichž se zkoumá integrita dat a

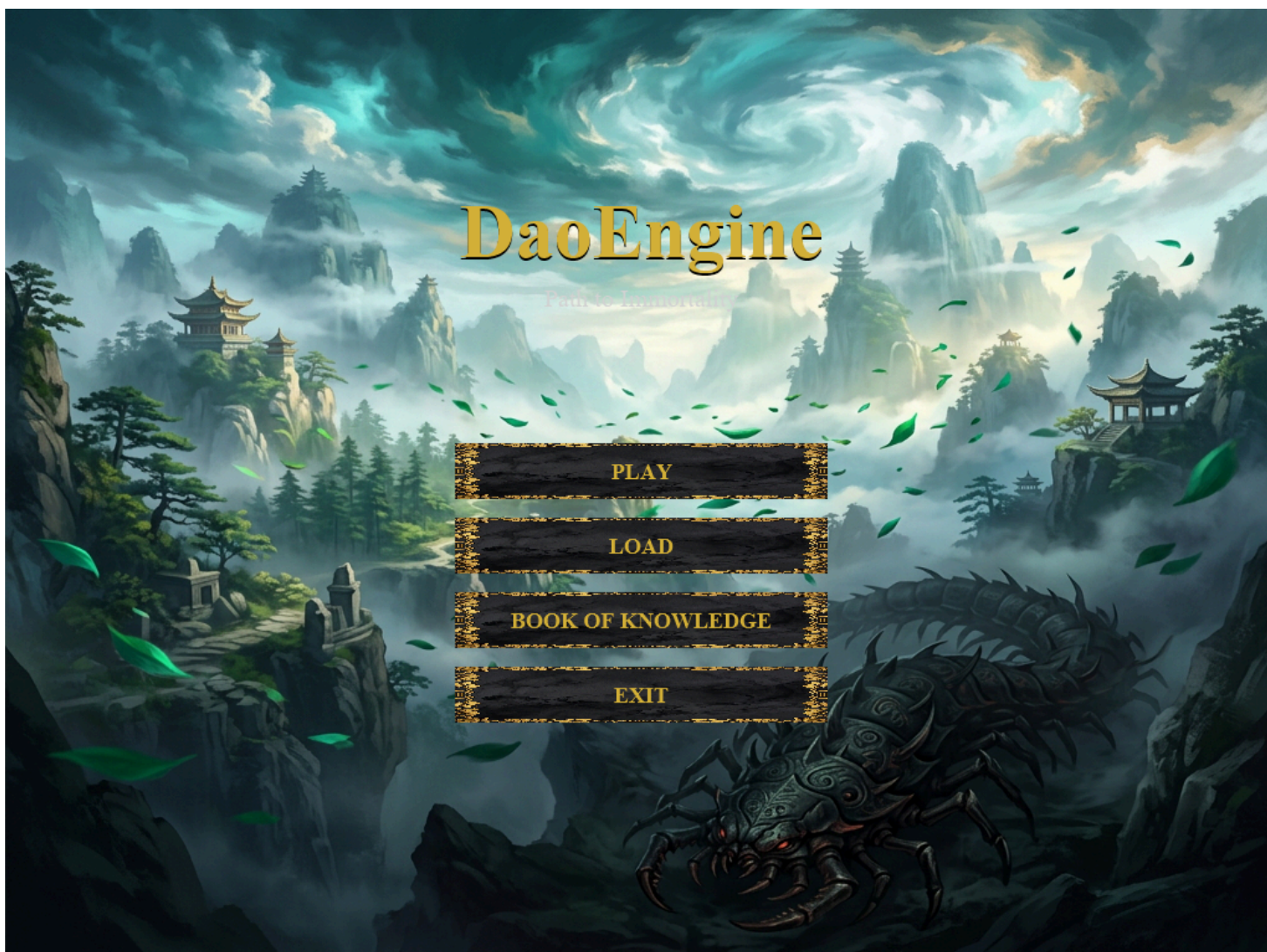
správnost reakcí na vstupy uživatele.

2.1.1 Klíčové komponenty SUT

1. **State Machine (Řízení stavů):** Zajišťuje striktní přechody mezi menu, hrou a koncovými stavy. Kritické pro testování životního cyklu aplikace.
2. **Combat & RPG Logic:** Matematické jádro počítající poškození na základě atributů jako Qi, HP a rezistence. Toto je hlavní oblast pro BVA (analýzu mezních hodnot).
3. **World Generator:** Algoritmus využívající procedurální pravidla pro tvorbu mapy. Testování se zde zaměřuje na validitu vygenerovaných dat a neprostupnost hranic mapy.
4. **Event System:** Mediátor distribuující události mezi entitami a questovým systémem.

2.2 Rozhraní aplikace a uživatelské prostředí

Uživatel interaguje s engineem prostřednictvím klávesnice (pohyb, interakce) a myši (útok, UI). Celé rozhraní je postaveno na vrstvách vykreslovaných přímo na Canvas.



Obrázek 1: Hlavní menu hry se správou uložených pozic.

- **Vstupy:** Klávesové zkratky [W,A,S,D] , [E] pro interakci, LMB pro bojové akce.
- **Výstupy:** Dynamické vykreslování v rozlišení 1024x768, HUD overlay s ukazateli HP a Qi.



Obrázek 2: Ukázka aktivního herního rozhraní (HUD) a quest trackeru.

2.3 Dynamické osvětlení a vizuální efekty

DaoEngine obsahuje pokročilý systém osvětlení, který simuluje atmosféru světa, změny času a speciální události jako je "Tribulace". Tento systém je kritický pro testování výkonu a správnosti renderování při extrémních světelných změnách.



Obrázek 3: Vizuální efekt blesku (Lightning Strike) během speciální události.

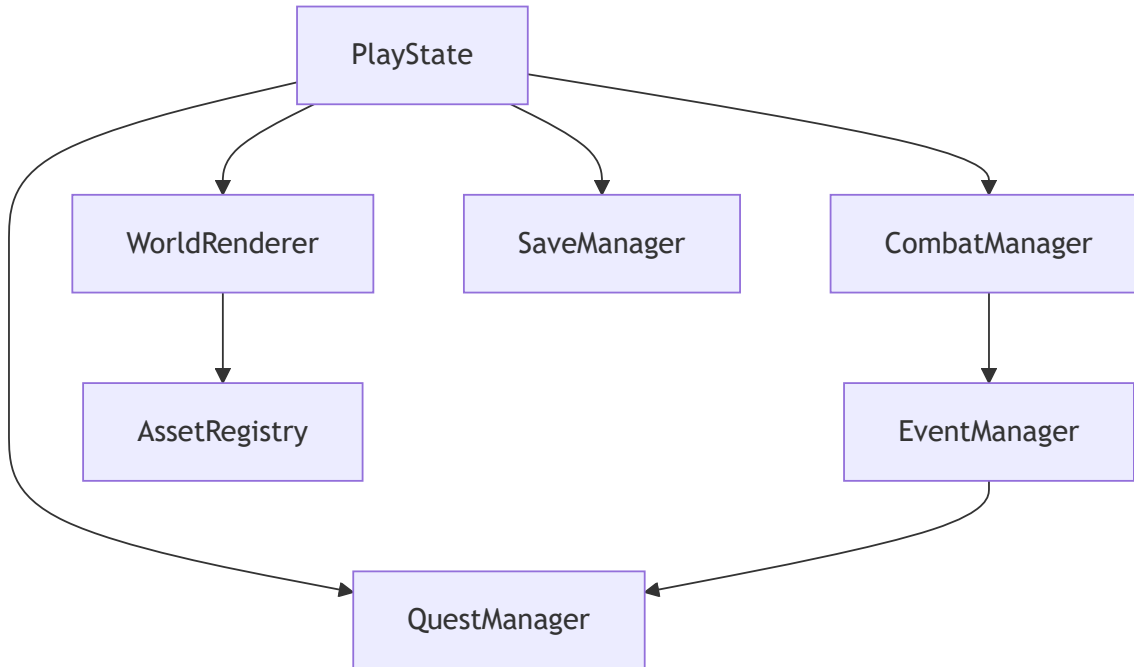
2.4 Identifikace rizikových oblastí

Při analýze SUT byly identifikovány následující kritické body, na které se testy zaměřují:

- **Přetečení atributů:** Riziko nekorektních výpočtů při extrémních hodnotách Qi nebo HP.
- **Zacyklení generátoru:** Možnost vzniku nevalidních cest v procedurální mapě.
- **Konzistence stavů:** Riziko uváznutí v nedefinovaném stavu při rychlém přepínání mezi UI okny.
 - **Mosty (typ 5):** Mosty jsou inteligentně umísťovány v rámci generování řek pomocí metody `drawBridgeSafe`, aby nedocházelo k jejich překrývání s jezery.
- 4. **Fáze 4: Dekorativní šum (typ 4):** Na zbývající volné dlaždice trávy je aplikován šum s 5% pravděpodobností, který umístí dekorativní prvky (kamínky, tráva).
- 5. **Fáze 5: Strategické rozmístění entit:** Portál `GateOfRealms` a důležitá NPC jsou umísťována algoritmem tak, aby byla zajištěna minimální vzdálenost (`centerThreshold`) od startovní pozice hráče.

2.4 Architektura tříd

Pro přehlednost je zde uvedeno schéma komunikace mezi hlavními systémy:



2.5 Dynamické objekty a systém entit

V DaoEnginu jsou objekty rozděleny na statické dlaždice mapy a dynamické entity, které disponují vlastním stavem a logikou aktualizace.

2.5.1 Entita hráče (Player)

Hráč je centrální entitou, která je definována nejen pozicí, ale i komplexní sadou dynamických atributů:

- **Kultivační statistiky:** Aktuální množství Qi, maximální kapacita a rychlost regenerace.
- **Bojové schopnosti:** Hráč může meditovat (klávesa Space) pro rychlou obnovu zdrojů nebo využívat aktivní techniky (např. *Fiery Palm*).
- **Interakční rádius:** Hráč automaticky detekuje blízké interaktabilní objekty v okruhu 150 pixelů (dle `game_config.json`).

2.5.2 Bestiář (Nepřátelé)

Nepřátelé využívají *Pathfinding (A)** a jejich chování je řízeno daty z `enemy_configs.json`:

- **Lesser Spirit Bat:** Rychlá entita (Speed 1.5), útočí v rojích, nízké HP (20).
- **Corpse Wight:** Pomalý strážce (Speed 0.6) s vysokým HP (60) a ničivým poškozením (15).
- **Spectral Archer:** Distanční jednotka, která udržuje odstup a využívá projektily typu *FIREBALL*.

2.6 Správa předmětů a inventáře

Systém předmětů je plně datově orientovaný a rozdělený do funkčních kategorií:

1. **Léčivé a podpůrné pilulky:** Např. *Healing Pill* (obnova 40 HP) nebo *Spirit Pill* (obnova 20 Qi).
2. **Průlomové (Breakthrough) pilulky:** Klíčové předměty pro postup hrou, jako je *Foundation Establishment Pill* nebo *Golden Core Pill*, které umožňují hráči postoupit na vyšší kultivační stupeň.
3. **Zbraně a artefakty:** Např. *Flying Sword* (ovládaný myslí) nebo *Fireball Staff*. Každá zbraň má v `weapon_configs.json` definovanou rychlost útoku, typ projektilu a spotřebu Qi.
4. **Knihy technik (SkillBookItem):** Speciální předměty, které po použití trvale odemknou hráči novou schopnost (např. *Manual: Void Sword*).
5. **Crafting materiály:** Suroviny jako *Iron Ore*, *Jade Leaf* nebo *Dragon Bone*, které slouží k výrobě artefaktů.

2.7 Přechody mezi světy a perzistence

Na rozdíl od serverové komunikace popsané v jiných projektech, DaoEngine řeší propojení map skrze **systém dimenzí a lokální perzistenci**.

2.7.1 Mechanika Realm Tokenu

Pro přechod do vyšších dimenzí přes `GateOfRealms` musí hráč vlastnit předmět **Realm Token**. Tento mechanismus zajišťuje lineární progresi a nutí hráče k interakci se světem před postupem dál.

2.7.2 Detailní struktura uložení (SaveData)

Při uložení hry (`SaveManager`) se do JSON souboru serializuje kompletní snapshot:

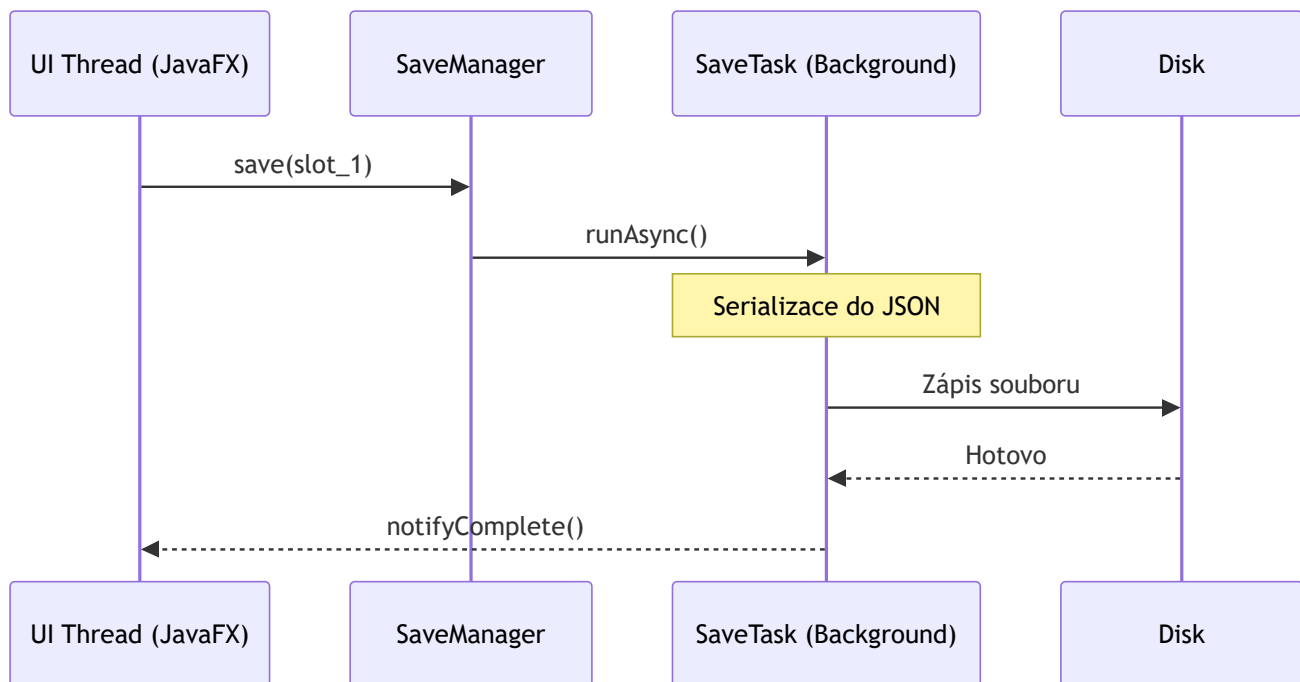
- **Stav světa:** Aktuální seed, biome a čas simulace.
- **Stav postavy:** Koordináty, HP, Qi, kultivační index a seznam naučených skillů.
- **Inventář:** ID všech držených předmětů a jejich množství.

2.8 Vnitřní běh programu (Sekvenční diagramy)

Pro lepší pochopení vnitřní logiky jsou zde znázorněny dva klíčové procesy:

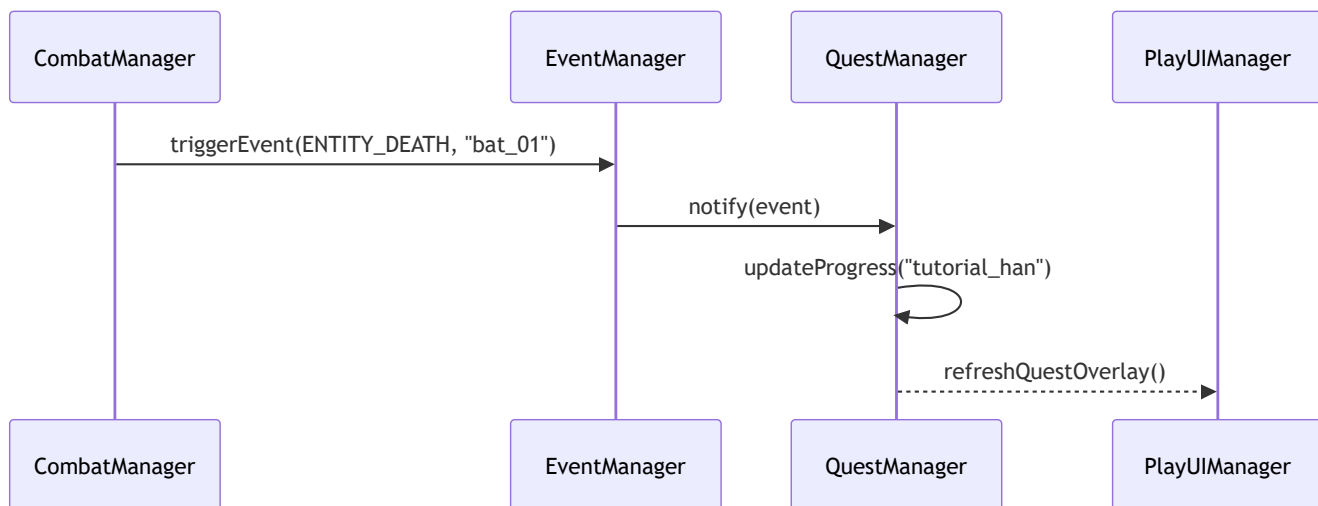
2.8.1 Asynchronní ukládání (Multithreading)

Tento proces zajišťuje, že zápis velkých JSON souborů nezpůsobí zásek obrazu:



2.8.2 Interakce souboje a úkolů (Event-Driven)

Ukázka toho, jak systém reaguje na smrt nepřítele bez přímé závislosti tříd:



2.9 Systém úkolů (Quest System)

Úkoly jsou definovány v `quests.json` a podporují různé typy cílů:

- **KILL Questy:** Např. *Clearing the Valley* (zabití 5 netopýrů).
- **COLLECT Questy:** Např. *Elder Mo's Catalyst* (nasbírání 10 kusů železné rudy).
- **Odměny:** Systém automaticky přiděluje předměty a bonusovou Qi po splnění všech podmínek.

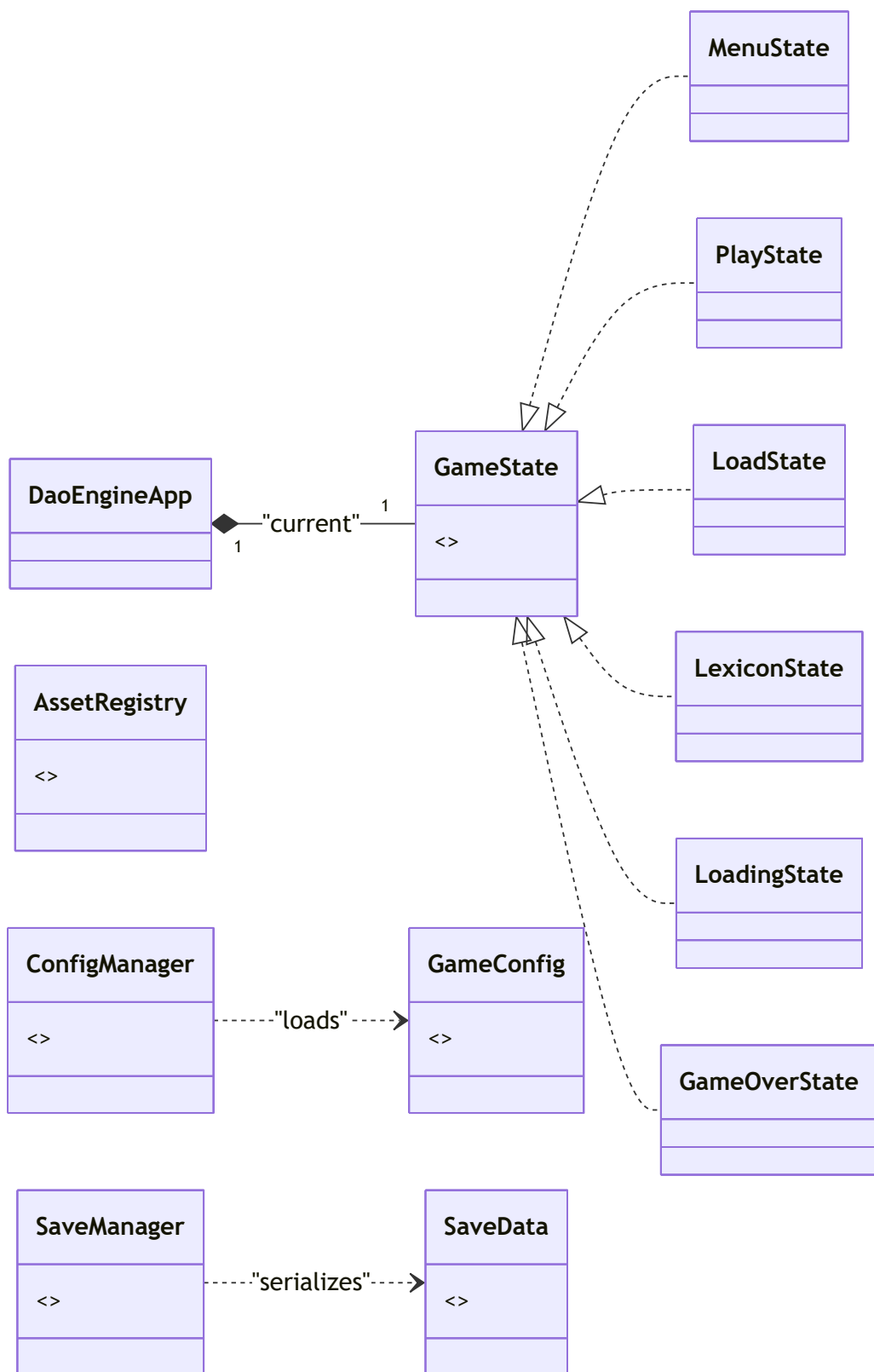
3 Strategie testování

3.1 Objektový model aplikace

Následující diagram znázorňuje kompletní architekturu DaoEngine a ukazuje vztahy mezi všemi hlavními třídami a subsystémy. Tento model slouží jako základ pro identifikaci kritických míst při návrhu testů.

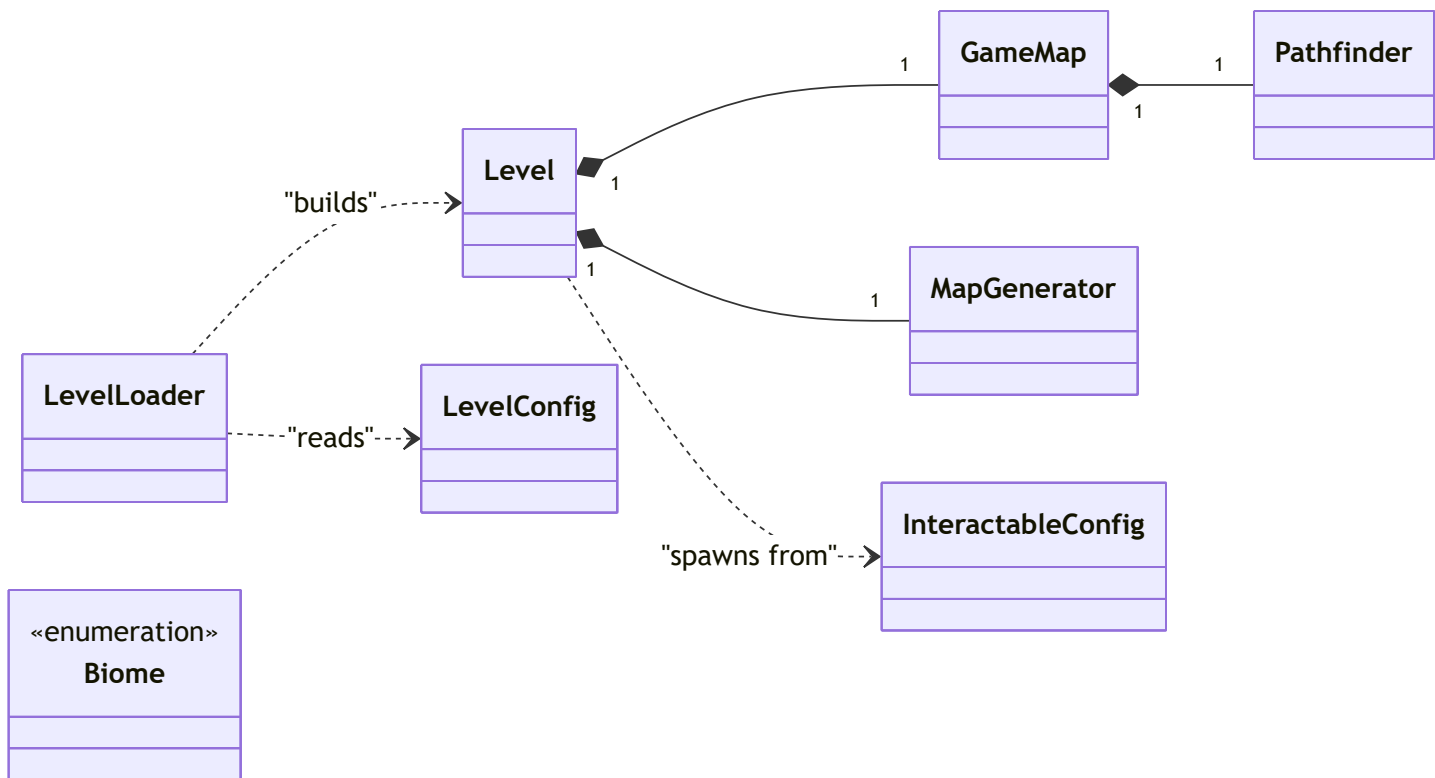
3.1.1 Jádro a správa stavů (Core & States)

Základní infrastruktura enginu, persistence a orchestrace všech stavů.



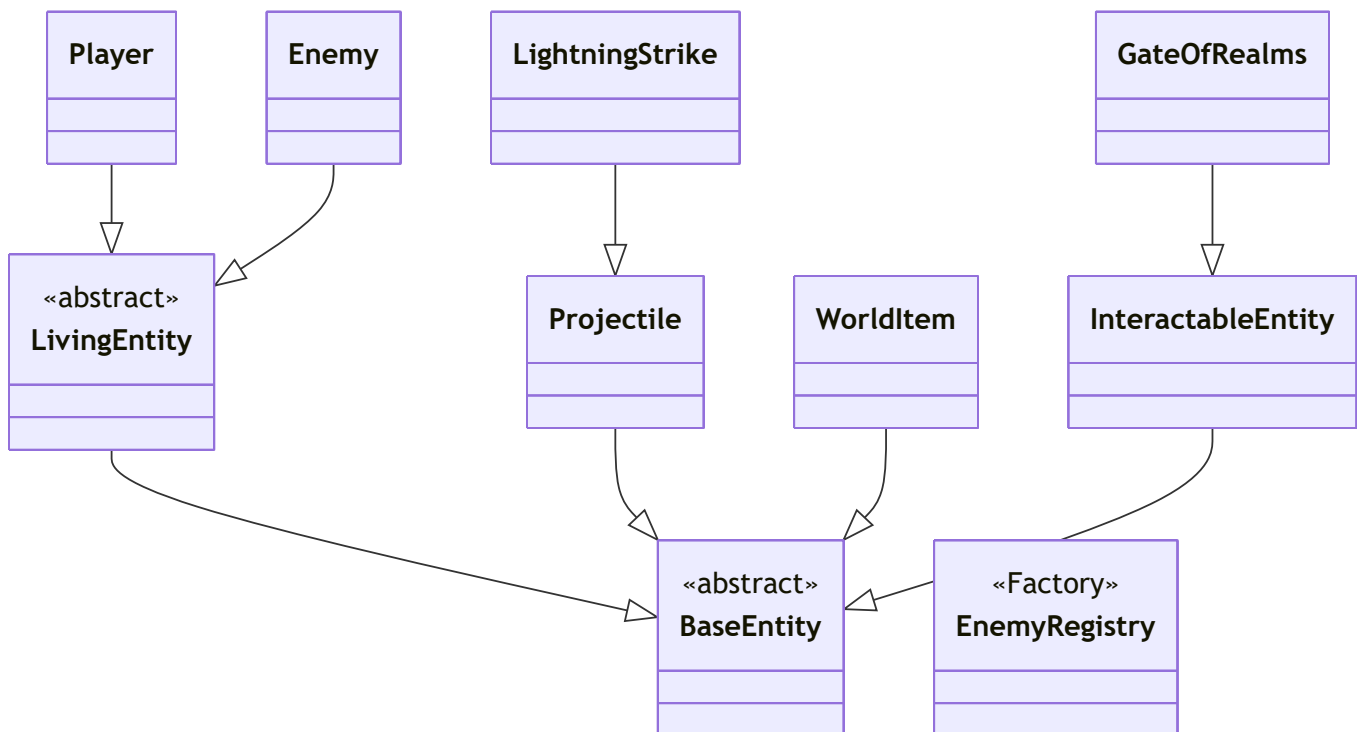
3.1.2 Architektura světa a navigace (World & Navigation)

Vše související s mapou, jejím generováním a hledáním cest.



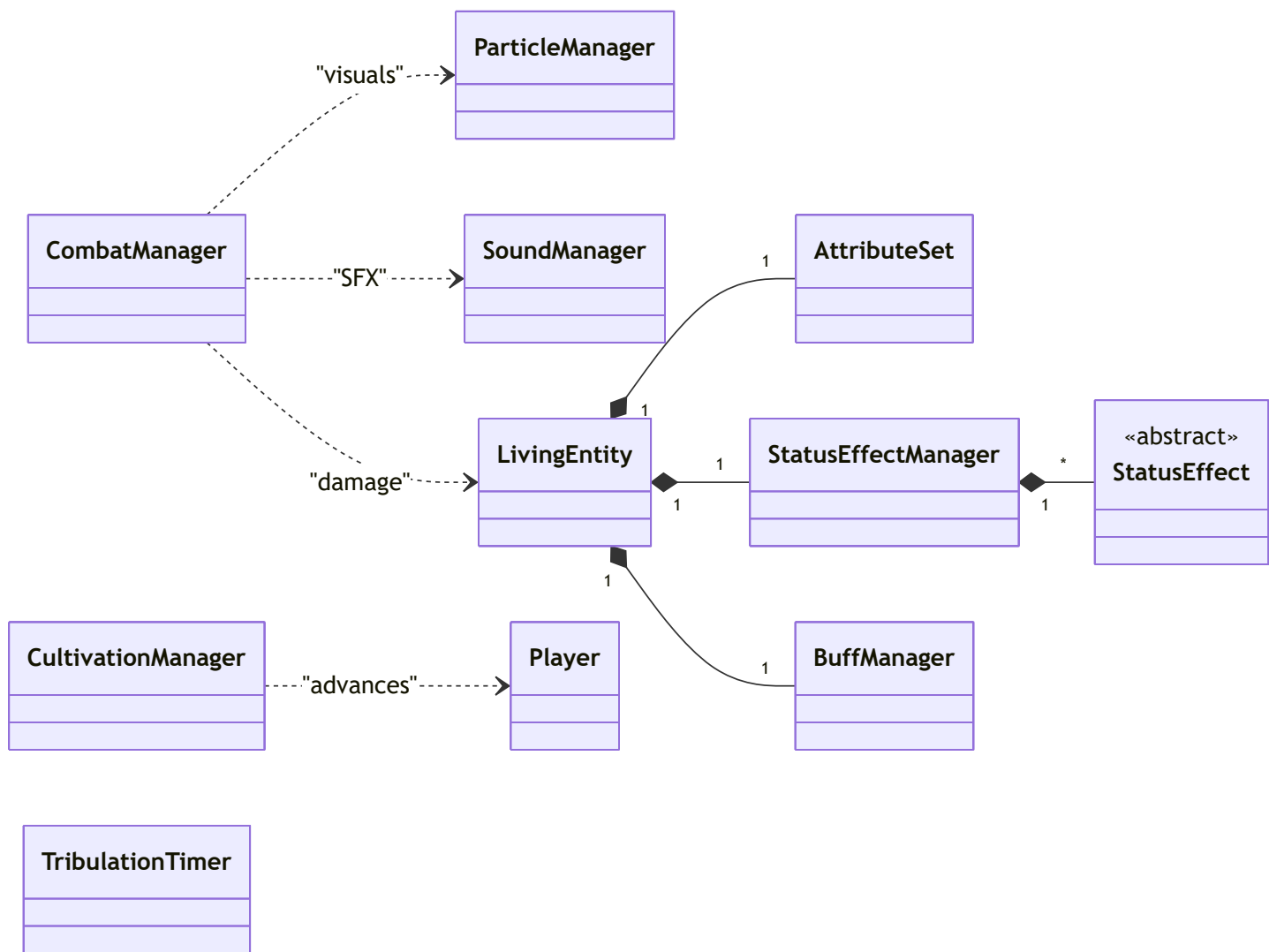
3.1.3 Kompletní hierarchie entit (Entities)

Všechny objekty, které se fyzicky vyskytují v herním světě.



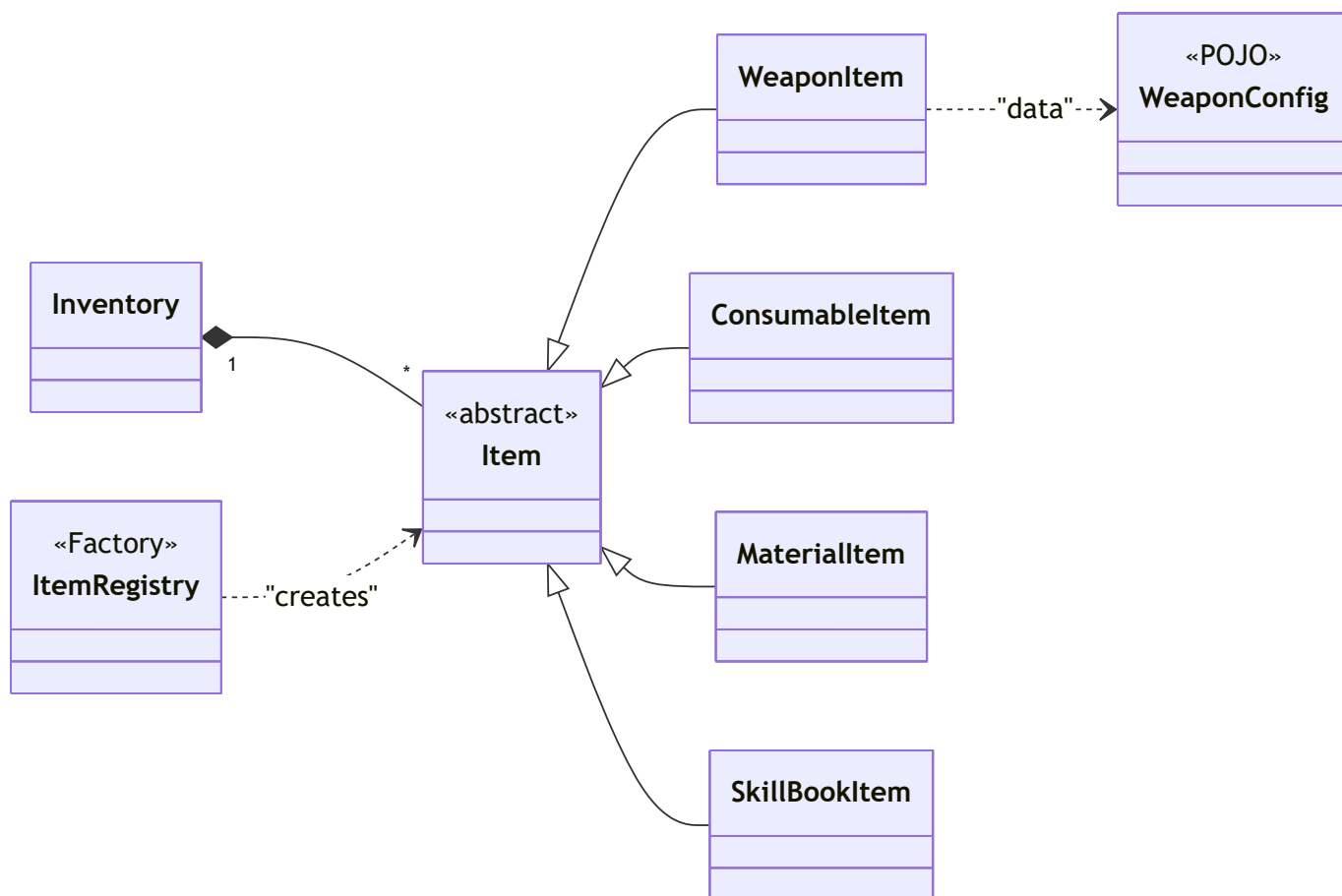
3.1.4 RPG Systém a bojová logika (RPG Logic)

Výpočetní jádro pro statistiky, efekty a souboj.



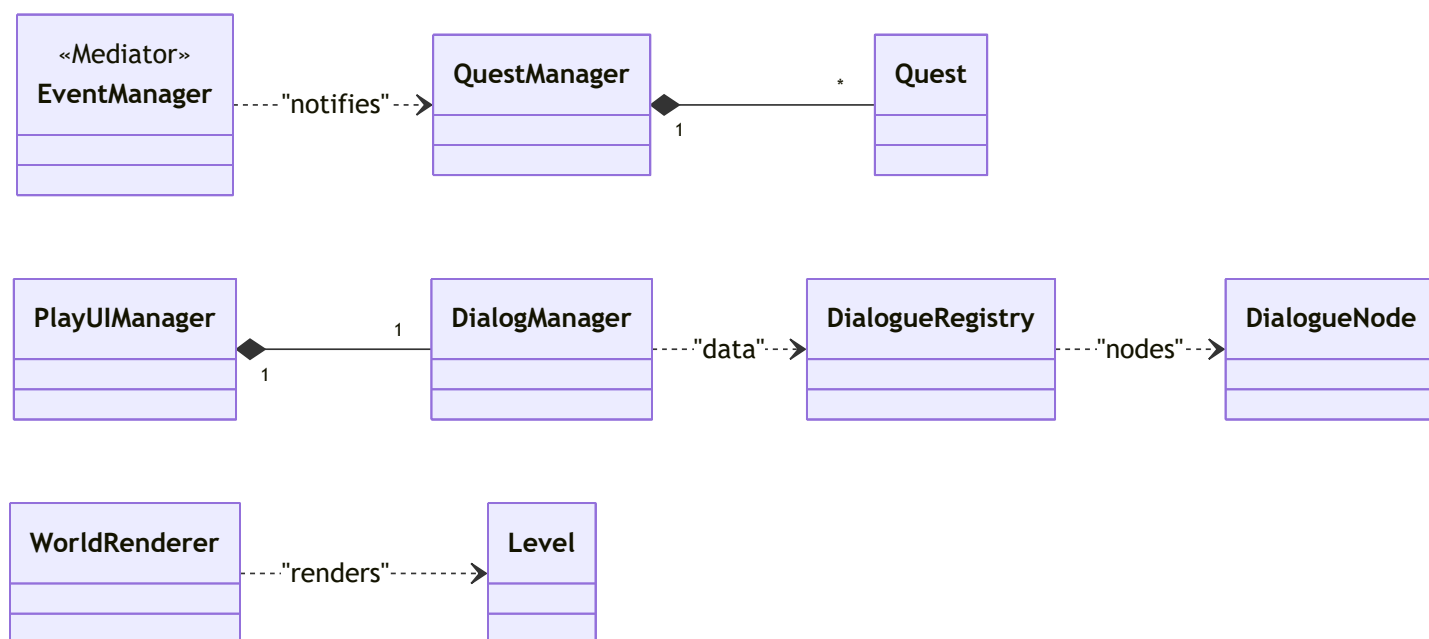
3.1.5 Systém předmětů a konfigurace (Items)

Typy předmětů, inventář a jejich datové konfigurace.



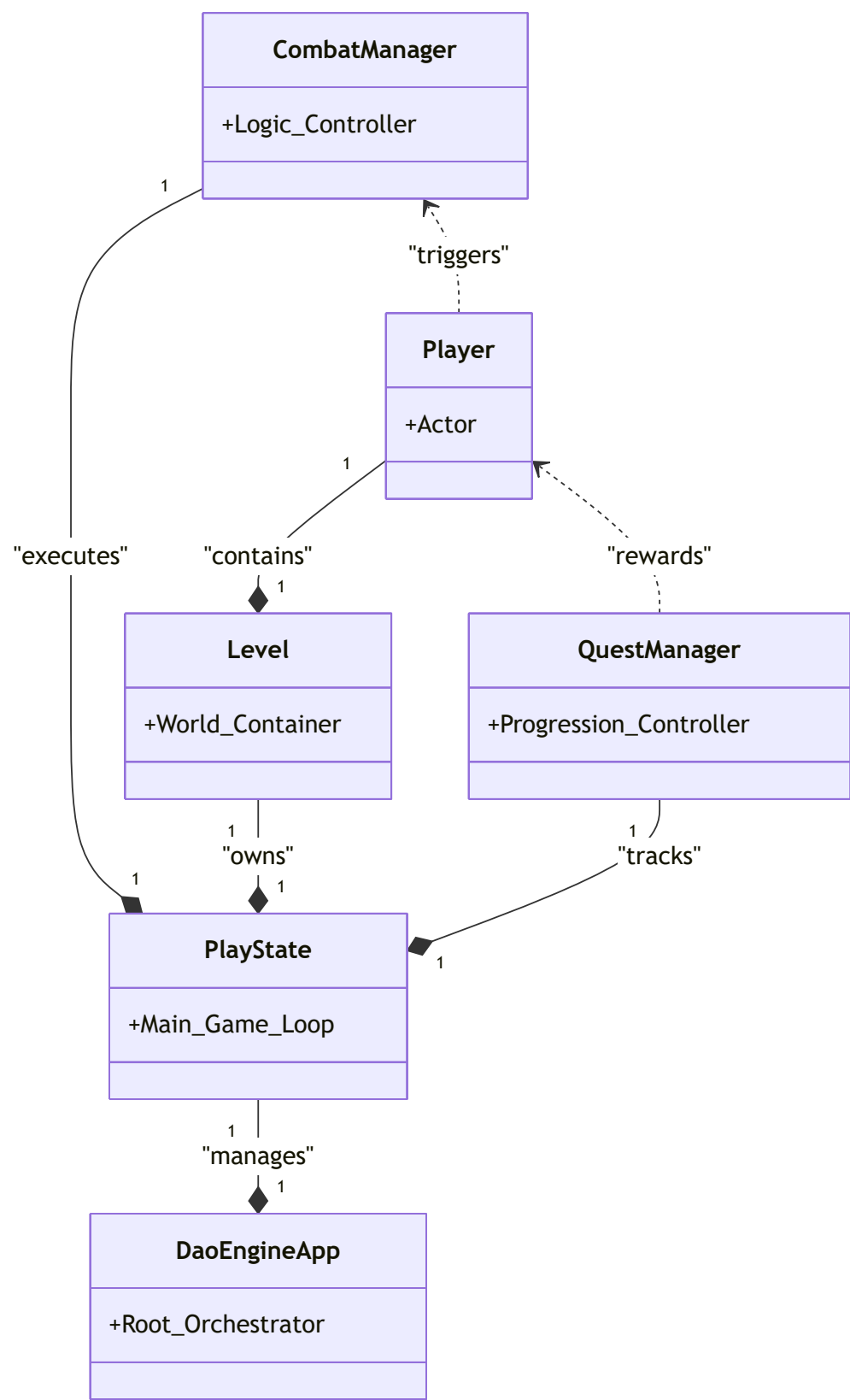
3.1.6 Komunikace, UI a události (Events & UI)

Zpracování událostí a vykreslování uživatelského rozhraní.



3.1.7 Integrate subsystémů (High-Level Integration)

Tento diagram ukazuje, jak se všechny hlavní komponenty propojují v běžícím enginu a vytvářejí herní zážitek.



3.2 Priorita částí aplikace (Prioritizace dle subsystémů)

Z komplexního modelu architektury vyplývá, že kritické body selhání se nacházejí v místech, kde dochází k orchestraci více subsystémů. Testování je prioritizováno následovně:

1. **Core & State Management (DaoEngineApp , GameState):**
 - **Priorita: Kritická.** Jedná se o "tep srdce" celé aplikace. Pokud selže přepínání stavů (např. z GameState do GameOverState), aplikace se stává neovladatelnou.
2. **Persistence Layer (SaveManager , SaveData):**
 - **Priorita: Kritická.** Zajišťuje integritu hráčova postupu. Selhání v serializaci nebo deserializaci objektu SaveData znamená nevratnou ztrátu herních dat.
3. **RPG Logic Core (CombatManager , AttributeSet , CultivationManager):**
 - **Priorita: Vysoká.** Jádro herní logiky. Veškeré RPG prvky (výpočet poškození, regenerace Qi, postup na vyšší ranky) stojí na matematické přesnosti těchto tříd.
4. **Event Driven Core (EventManager , GameEventListener):**
 - **Priorita: Vysoká.** Funguje jako "lepidlo" celého systému. Zajišťuje, že události v souboji (smrt nepřítele) správně vyvolají reakci v jiných subsystémech (např. v QuestManageru).
5. **World Architecture (MapGenerator , LevelLoader):**
 - **Priorita: Střední.** Komplexní procedurální generování vyžaduje testování mezních stavů, aby nedocházelo k vytváření nehratelných map (zablokované cesty).
6. **Item System (Inventory , ItemRegistry):**
 - **Priorita: Střední.** Zajišťuje správnou správu herních objektů. Chyba v ItemRegistry by mohla vést k načtení nevalidních assetů nebo pádu aplikace při interakci.

3.3 Přehled rizik (Dle architektonických subsystémů)

Při analýze komplexního modelu bylo identifikováno 10 klíčových rizik, rozdělených dle jejich původu v subsystémech:

ID	Subsystém	Popis rizika	Dopad na uživatele
R01	Persistence	Poškození JSON souboru při zápisu (SaveManager)	Úplná ztráta postupu hrou.
R02	Core	Race condition v asynchronním SaveTask	Pád aplikace při paralelním přístupu k datům.
R03	Infrastructure	Chybné parsování GameConfig nebo LevelConfig	Aplikace se nespustí (Fatal Error).
R04	World	Nekonečná smyčka v Pathfinder (A*)	Zámrz aplikace při výpočtu cesty nepřítele.

ID	Subsystem	Popis rizika	Dopad na uživatele
R05	RPG Logic	Přetečení (Overflow) v AttributeSet (HP/Qi)	Nesmrtelnost entit nebo okamžitá smrt.
R06	Event Core	Nedoručení GameEvent do QuestManageru	Nemožnost splnit úkol (zaseknutý progres).
R07	Infrastructure	Únik paměti v AssetRegistry při střídání levelů	Postupné zpomalování aplikace (OOM Error).
R08	World	Prostupnost terénu v GameMap (Wall Clipping)	Postava se zasekne v textuře zdi.
R09	RPG Logic	Chyba v TribulationTimer (Blesky)	Hráč dostane poškození v nesprávný čas.
R10	Item System	Neplatné ID v ItemRegistry	NullPointerException při pokusu o sebrání itemu.

3.4 Určení priority (Risk Matrix)

Priorita testování je určena vztahem mezi **možným poškozením (Impact)** a **pravděpodobností selhání (Probability)**.

	High (Pravděpodobnost)	Medium	Low
High (Dopad)	A (Kritické)	B (Vysoké)	C (Střední)
Medium	B (Vysoké)	B (Vysoké)	C (Střední)
Low	C (Střední)	C (Střední)	C (Střední)

- **Priorita A (Kritická):** Persistence , Core , RPG Logic .
- **Priorita B (Vysoká):** Event-Driven Core , World Architecture .
- **Priorita C (Střední):** UI & Visuals , Item System .

3.5 Charakteristika – Funkcionality

Hlavní funkcionality určené k hloubkovému testování:

- **F1:** Serializace a deserializace stavu světa (SaveManager).
- **F2:** Navigace a detekce kolizí (Pathfinder + GameMap).
- **F3:** Komunikace skrze EventManager (Pub/Sub pattern).
- **F4:** Matematická logika souboje (CombatManager + AttributeSet).

3.6 Test Levels a mapování rizik

Tabulka níže mapuje procesy na rizika a určuje úroveň intenzity testování pro jednotlivé fáze.

Proces	Subsystem	Třída rizika	Revize	Vývojářské testy	Systémové testy
Inicializace	Infrastructure	C	ano	vysoké	střední
Změna stavu	State Management	A	ano	vysoké	vysoké
Běh světa	World / Entity	B	ano	střední	vysoké
Souboj	RPG Logic	A	ano	vysoké	střední
Eventy	Event Core	A	ano	střední	vysoké
Ukládání	Persistence	A	ano	vysoké	vysoké

3.7 Použité testovací techniky

Pro dosažení maximálního pokrytí rizik identifikovaných v sekci 3.3 budou aplikovány následující techniky:

- **Analýza tříd ekvivalence (EC):** Použita pro testování atributů entit (HP, Qi) a souřadnic na mapě. Cílem je zredukovat počet testů při zachování detekce chyb v logice.
- **Analýza mezních hodnot (BVA):** Aplikována na hranice mapy (indexy 0 a 99/249), limity inventáře a kritické stavy života (0 HP, 1 HP).
- **Párové testování (Pairwise):** Využito pro testování generátoru světa – kombinace různých **Biomů** (Forest, Ice, Fire) a **Seedů** (kladné, záporné, nula).
- **Procesní testování (Transition Testing):** Zaměřeno na stavový automat hry (MenuState -> PlayState -> Save -> Exit) a logiku plnění úkolů.

3.8 Testovací prostředí

Vzhledem k využití asynchronního zpracování a hardwarové akcelerace pro Canvas (JavaFX) je testování prováděno v následujícím prostředí:

- **Softwarové vybavení:**
 - Operační systém: Windows 10/11
 - Runtime: Java 17+ (OpenJDK)
 - Knihovny: JavaFX 17+, Jackson (JSON parsing)
- **Hardwarové nároky (Minimální):**
 - RAM: 4 GB (kritické pro generování velkých map 250x250)
 - GPU: Podpora HW akcelerace pro plynulé vykreslování osvětlení (60 FPS)

4 Přehled testovacích technik

Následující tabulka detailně specifikuje konkrétní testovací techniky aplikované na jednotlivé subsystémy DaoEngine a jejich realizaci v JUnit testech.

Subsystem	Proces / Podproces	Třída rizika	Vývojářské testy	Technika	Realizace v kódu
Persistence	Serializace uložení	A	vysoké	MCC, C/DC	SaveLoadIntegrationTest
Event Core	Doručení GameEvent	A	střední	JUnits, Procesní hloubka 2	QuestIntegrationTest
RPG Logic	Výpočet poškození	A	vysoké	MC/DC, BVA	CombatFlowIntegrationTest
RPG Logic	Správa atributů (HP/Qi)	A	vysoké	BVA, EC	AttributeSetTest
RPG Logic	Status efekty (Buffs)	B	střední	Transition Testing	StatusEffectManagerTest
World	Pathfinding (A*)	B	střední	Pairwise	CombatIntegrationTest
Infrastructure	Registrace assetů	C	nízké	CRUD extended	RegistryIntegrationTest

5 Testovací situace

V první řadě je třeba zmínit, že aplikace DaoEngine je navržena jako robustní herní prostředí. Uživatel nemá k dispozici žádná textová pole pro přímý vstup dat, což eliminuje riziko pádů na nevalidních řetězcích. Testování se proto zaměřuje na vnitřní logiku enginu, která obsahuje komplexní rozhodovací větve.

5.1 MCC – Multiple Condition Coverage

Zde testuji logiku **validace herního uložení** v `SaveManager`. Pro úspěšné načtení nesmí být soubor poškozen a zároveň musí existovat platný slot nebo být povolen přepis.

Podmínka: `!isCorrupted AND (slotExists OR overwriteEnabled)`

- **A:** !isCorrupted
- **B:** slotExists
- **C:** overwriteEnabled
- **R:** Výsledek (Možnost zápisu/načtení)

	1	2	3	4	5	6	7	8
A	1	1	1	1	0	0	0	0
B	1	1	0	0	1	1	0	0
C	1	0	1	0	1	0	1	0
R	1	1	1	0	0	0	0	0

5.2 MC/DC – Multiple Condition/Decision Coverage

Zde testuji podmínku pro **detekci kolize s hranicí mapy**. Tato logika je kritická pro stabilitu pohybu všech entit.

Podmínka: (isMoving OR isMeditating) AND (outLeft OR outRight OR outTop OR outBottom)

	1	2	3	4	5	6	7	8	9	10	11	12
isMoving	1	0	1	0	0	0	0	0	0	0	0	0
isMeditating	0	1	0	1	0	0	0	0	0	0	0	0
outLeft	1	0	1	0	0	0	1	0	0	0	0	0
outRight	0	1	0	1	0	0	0	1	0	0	0	0
outTop	1	0	0	0	1	0	1	0	0	0	0	0
outBottom	0	1	0	0	0	1	0	1	0	0	0	0
Výsledek	1	1	1	1	0	0	0	0	0	0	0	0

5.3 Pairwise testing

Testování kombinací parametrů **MapGeneratoru** pro zajištění stability generování světa.



5.3.1 Vstupní hodnoty

Vstupy	Třídy ekvivalence	Priorita
X (Pozice)	≤ -1 , mezi (0-99) , ≥ 100	A
Y (Pozice)	≤ -1 , mezi (0-99) , ≥ 100	A
Biome	FOREST (0) , ICE (1) , FIRE (2)	A
Seed	< 0 , 0 , > 0	B
VeinCount	0 , 10 , 100	B

5.3.2 Testy (Pairwise Cases)

case	X	Y	Biome	Seed	VeinCount	pairings
1	≤ -1	≤ -1	FOREST	< 0	0	10
2	≥ 100	≥ 100	ICE	> 0	0	10
3	≤ -1	≥ 100	FIRE	0	10	8
4	≥ 100	≤ -1	FOREST	> 0	10	8
5	≤ -1	≤ -1	ICE	0	100	6
6	≥ 100	≥ 100	FIRE	< 0	100	6
7	≤ -1	≥ 100	FOREST	> 0	0	4
8	≥ 100	≤ -1	ICE	< 0	10	4
9	≤ -1	≤ -1	FIRE	> 0	10	4
10	≥ 100	≥ 100	FOREST	0	10	4
11	≤ -1	≥ 100	ICE	< 0	0	4
12	≥ 100	≤ -1	FIRE	> 0	100	4

case	X	Y	Biome	Seed	VeinCount	pairings
13	<= -1	<= -1	FOREST	< 0	10	4
14	>= 100	>= 100	ICE	> 0	0	4
15	<= -1	>= 100	FIRE	0	10	4
16	>= 100	<= -1	FOREST	< 0	100	4

5.4 C/DC – Condition/Decision Coverage

Otestování **logiky Quest systému** při odevzdání úkolu.

Podmínka: !isNpcAggro AND (hasQuestItems OR hasKilledTarget)

Testy:

- 0 1 0 = 1 (NPC není agresivní, máme itemy)
- 1 0 1 = 0 (NPC je agresivní, nepustí nás k dialogu)

5.5 CC – Condition Coverage

Logika **interakce s prostředím** (např. meditace na Spirit Vein).

Podmínka: isStandingOnVein AND isPressingSpace AND hasEnergyMissing

Testy:

- 0 1 1 = 0
- 1 1 0 = 0
- 1 1 1 = 1 (Vše splněno, začíná regenerace)

5.6 DC – Decision Coverage

Vytvoření nového **vizuálního efektu (Particle)** při zásahu.

Podmínka: effectsEnabled AND onscreen AND particleCount < 1000

Testy:

- 1 1 1 = 1
- 1 0 1 = 0

5.7 Ruční testování UI panelu

Testování hlavního herního HUDu a interakcí.

- **Hlavní pozitivní průchod:** Start -> Pohyb k Vein -> Meditace [Space] -> Útok na nepřítele -> Sebrání Lootu -> Otevření inventáře -> Konec.
- **Nejčastější průchod:** Start -> Pohyb -> Interakce s NPC [E] -> Přijetí Questu -> Boj.

5.8 Hlubková analýza datové integrity (SaveData)

V této sekci je provedena detailní analýza všech perzistentních atributů třídy `SaveData` pomocí tříd ekvivalence (EC) a mezních hodnot (BVA), aby byla zajištěna 100% integrita při ukládání a načítání postupu hry.

5.8.1 Analýza ekvivalentních tříd (EC) pro strukturu uložení

A. Základní metadata a konfigurace

Pole	Třída (EC)	Rozsah / Hodnota	Typ	Očekávané chování
mapSeed	EC01	Libovolné long	Platná	Generování identického světa.
	EC02	Text / null	Neplatná	Selhání parseru (IOException).
biome	EC03	FOREST, ICE, FIRE	Platná	Načtení příslušné palety assetů.
	EC04	"" , null , DESERT	Neplatná	Fallback na defaultní FOREST.
levelConfigPath	EC05	Cesta k JSONu	Platná	Načtení parametrů mapy.
	EC06	Neexistující cesta	Neplatná	Pád na <code>FileNotFoundException</code> .
currentTime	EC07	0.0 až 80.0	Platná	Čas v rámci tribulačního cyklu.
	EC08	> 80.0	Platná	Čas po skončení tribulace.

B. Statistiky a stav hráče

Pole	Třída (EC)	Rozsah / Hodnota	Typ	Očekávané chování
hp / maxHp	EC09	0.0 < hp <= maxHp	Platná	Hráč je naživu.
	EC10	hp <= 0.0	Platná	Hráč se načte mrtv (respawn).
	EC11	hp > maxHp	Neplatná	HP oříznuto na <code>maxHp</code> .
qi / maxQi	EC12	0.0 <= qi <= maxQi	Platná	Správný stav energie.
	EC13	qi < 0.0	Neplatná	Qi resetováno na 0.0.
playerX / Y	EC14	0.0 až 3200.0	Platná	Hráč v rámci mapy (pro 100x100).
	EC15	< 0.0 nebo > 3200.0	Neplatná	Hráč se načte v prázdnosti.
activeHotbar	EC16	{0, 1, 2, 3, 4}	Platná	Výběr aktivního slotu.

Pole	Třída (EC)	Rozsah / Hodnota	Typ	Očekávané chování
	EC17	< 0 nebo > 4	Neplatná	Reset na slot 0.

C. Inventář a Progres

Pole	Třída (EC)	Rozsah / Hodnota	Typ	Očekávané chování
inventory	EC18	Seznam o 30 prvcích	Platná	Načtení ID předmětů.
	EC19	Seznam délky != 30	Neplatná	Oříznutí nebo pád (IndexError).
completedQuests	EC20	Seznam ID questů	Platná	Ochrana proti opakování questů.
worldFlags	EC21	Mapa {String:Bool}	Platná	Načtení progresu příběhu.

5.8.2 Analýza mezních hodnot (Boundary Value Analysis - BVA)

Detailní analýza hraničních hodnot pro všechny kritické číselné atributy v systému perzistence.

Atribut	Mez	Hodnota	Testovaný stav
mapSeed	Minimum	Long.MIN_VALUE	Test záporného seedu.
	Maximum	Long.MAX_VALUE	Test maximálního kladného seedu.
hp	Smrt	0.0	Okamžitý respawn po načtení.
	Těsně nad	0.1	Hráč přežije "o vlásek".
	Max	100.0 (maxHp)	Hráč je zcela fit.
qi	Prázdko	0.0	Hráč nemůže kouzlit.
	Plno	100.0 (maxQi)	Hráč je plný energie.
playerX / Y	Původ	0.0	Pozice v levém horním rohu.
	Konec	3200.0	Pozice v pravém dolním rohu.
activeHotbar	První	0	První slot hotbaru.
	Poslední	4	Pátý slot hotbaru.
currentTime	Začátek	0.0	Právě začal cyklus.
	Limit	80.0	Hrana pro spuštění tribulace.

5.8.3 Specifické Pairwise scénáře (Kombinace dat)

Kombinace různých typů dat pro odhalení skrytých závislostí mezi perzistencí a herní logikou.

ID	HP (P1)	Biome (P2)	Tribulace (P3)	Výsledek
T01	0.0	FIRE	1 (Ano)	Smrt v momentě tribulace v lávě.
T02	100.0	ICE	0 (Ne)	Standardní bezpečné uložení.
T03	-5.0 (Inv)	FOREST	0 (Ne)	Oprava HP na 0.0 během načítání.
T04	50.0	FIRE	1 (Ano)	Boj v tribulaci po načtení.
T05	100.0	null (Inv)	0 (Ne)	Fallback biomu na FOREST.

6 Analýza vstupů

V této sekci analyzujeme data, která do systému DaoEngine vstupují z externích zdrojů (konfigurační soubory) a skrze uživatelské rozhraní.

6.1 Vstupy z konfiguračních souborů (JSON Configs)

Jedná se o statické vstupy načítané při startu aplikace skrze knihovnu Jackson (`ConfigManager`). Tyto objekty jsou klíčové pro správnou inicializaci herního světa a entit.

Očekávané parametry (při spuštění):

1. **Hráčské statistiky (Integery/Doubles):**
 - `initialMaxHp` (validní rozsah > 0)
 - `initialMaxQi` (validní rozsah >= 0)
 - `baseSpeed` (v rozmezí 0.5 až 5.0)
 - `size` (pevných 32px pro správnou detekci kolizí)
2. **Konfigurace mapy (`mapX.json`):**
 - `width` , `height` (v rozmezí 100 až 250 dlaždic)
 - `seed` (libovolné parsovatelné long)
 - `biome` (validní řetězec: "forest", "ice", "fire")

Veškeré nečíselné hodnoty v polích očekávajících čísla vedou k chybě `MismatchedInputException` a okamžitému pádu aplikace (Fail-fast mechanismus). Desetinná čísla v celočíselných polích jsou automaticky zaokrouhlena knihovnou Jackson, což je sice nežádoucí, ale engine to dokáže zpracovat bez pádu.

6.2 Uživatelské vstupy (Herní interakce a GUI)

Jedná se o dynamické vstupy realizované hráčem během aktivního hraní skrze klávesnici a myš.

- Výběr aktivního předmětu (Hotbar):**

- Vstup: Klávesy 1 až 5 .
- Podmínka: Musí se jednat o validní index $0 \leq \text{index} < 5$. Ostatní klávesy jsou ignorovány.
- **Interakce s NPC a objekty:**
 - Vstup: Klávesa E .
 - Podmínka: `distance(player, object) <= interactionRadius` (standardně 150px dle `game_config.json`).
- **Meditace a obnova Qi:**
 - Vstup: Klávesa SPACE .
 - Podmínka: Hráč se nesmí hýbat a nesmí být v pauze.
- **Použití předmětu / Útok:**
 - Vstup: Levé tlačítko myši (Primary Click).
 - Podmínka: Vybraný slot nesmí být prázdný a `attackCooldown` musí být roven 0.

6.3 Systémové výstupy a perzistence (Save/Log)

Zde analyzujeme data, která simulátor vysílá ven pro zajištění perzistence a diagnostiky.

- **"RUN"**: Standardní běhový stav, periodický výpis do konzole o stavu entit (logování).
- **"SAVING"**: Ve chvíli, kdy hráč vyvolá uložení nebo dojde k automatickému uložení při přechodu mapy. Do souboru `save_slot_X.json` je vyexportován aktuální stav objektu `SaveData` .
- **"EXIT"**: Při zavření okna aplikace (JavaFX `onCloseRequest`). Simulátor uvolní grafické prostředky a bezpečně ukončí asynchronní vlákna (včetně `SaveTask`).

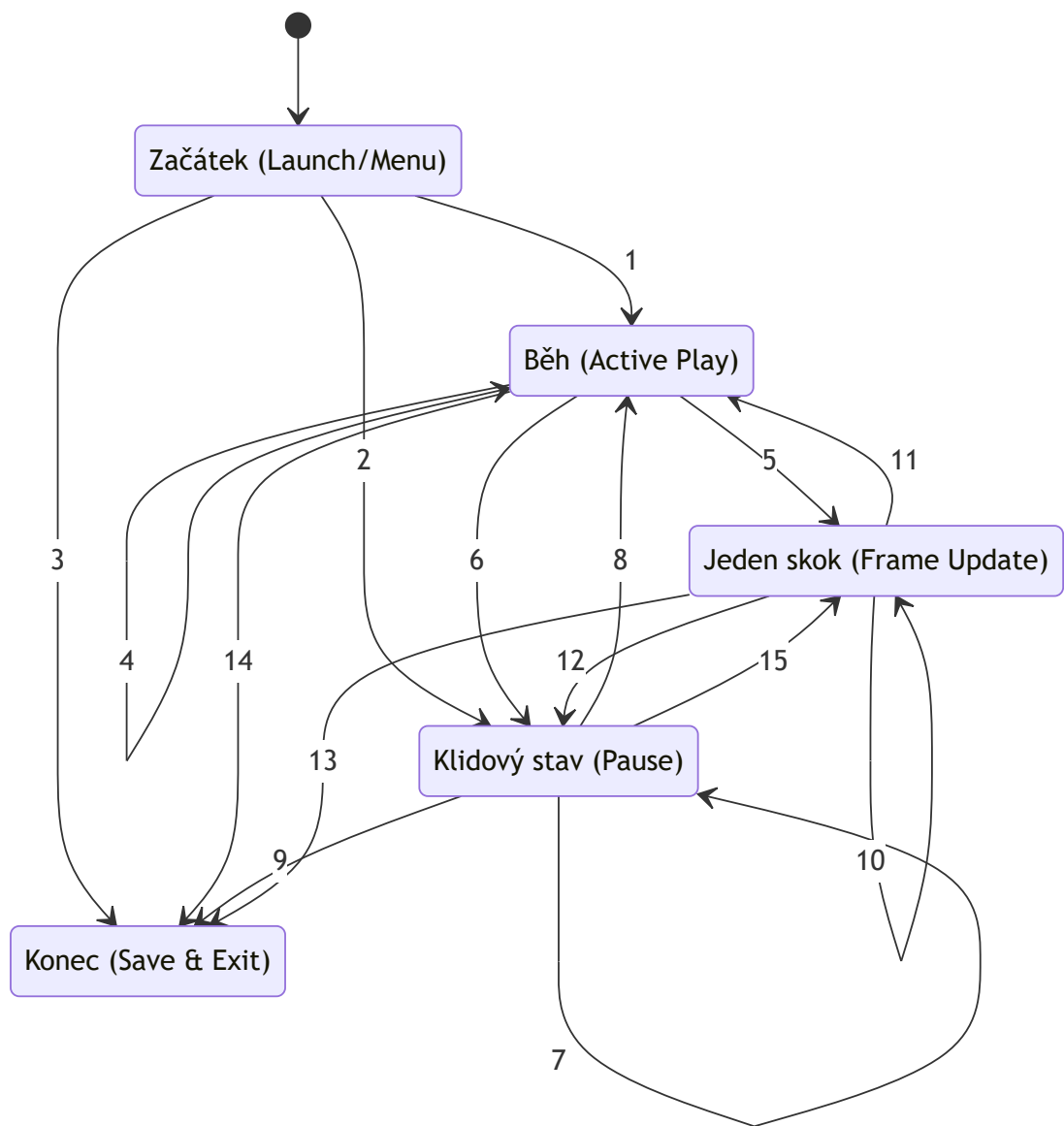
Veškeré nečekané stavy během zápisu (např. nedostatek místa na disku) vedou k zachycení výjimky a zobrazení varovného dialogu uživateli, aniž by došlo ke spadnutí hlavní herní smyčky.

7 Testování průchodu (Process Testing)

V této sekci analyzujeme chování aplikace DaoEngine jako stavového automatu. Cílem je ověřit, že přechody mezi hlavními stavy simulace (Menu, Běh, Pauza, Konec) probíhají korektně a nedochází k uvíznutí v nedefinovaném stavu.

7.1 Graf přechodů

Následující orientovaný graf znázorňuje možné stavy aplikace a hrany (přechody) mezi nimi. Graf simuluje životní cyklus herní session od spuštění po ukončení.



Mapování stavů na architekturu DaoEngine

Pro účely testování průchodu byl komplexní stavový stroj aplikace (viz sekce 3.1) abstrahován do 5 hlavních logických uzlů, které reprezentují klíčové fáze životního cyklu programu:

Stav v grafu	Reálná třída v DaoEngine	Funkční význam
Začátek	MenuState	Výchozí bod, inicializace UI a Assetů.
Běh	PlayState	Aktivní simulace světa, výpočet fyziky a AI.
Klidový stav	PauseState , LexiconState	Simulace je pozastavena, hráč interaguje s UI menu.
Jeden krok	LoadingState , LoadState	Krátkodobé stavy pro načítání levelu nebo synchronizaci dat.

Stav v grafu	Reálná třída v DaoEngine	Funkční význam
Konec	GameOverState / Program Exit	Finální stav po porážce nebo korektním ukončení.

7.2 Analýza hran (Uzly a přechody)

Uzly	Vstupové hrany	Výstupní hrany
Začátek	-	1, 2, 3
Běh	1, 4, 8, 11	4, 5, 6, 14
Klidový stav	2, 6, 7, 12	7, 8, 9, 15
Jeden krok	5, 10, 15	10, 11, 12, 13
Konec	3, 9, 13, 14	-

7.3 Hloubka pokrytí 1 (Edge Coverage)

Cílem je projít každou hranu grafu alespoň jednou pro zajištění základní funkčnosti všech přechodů.

7.3.1 Seznam kombinací

- **Začátek:** 1, 2, 3
- **Běh:** 4, 5, 6, 14
- **Klidový stav:** 7, 8, 9, 15
- **Jeden krok:** 10, 11, 12, 13

7.3.2 Testovací scénáře (Cesty)

- **T1:** 1 -> 4 -> 5 -> 12 -> 9
- **T2:** 2 -> 7 -> 8 -> 14
- **T3:** 2 -> 15 -> 10 -> 11 -> 6 -> 15 -> 13
- **T4:** 3 (Okamžité ukončení z menu)

7.4 Hloubka pokrytí 2 (Edge-Pair Coverage)

Cílem je otestovat všechny dvojice po sobě jdoucích hran pro odhalení chyb v logice následných akcí.

7.4.1 Výběr kombinací pro testy

- **Běh:** (1,4), (1,5), (1,6), (1,14), (4,4), (4,5), (4,6), (4,14), (8,4), (8,5), (8,6), (8,14), (11,4), (11,5), (11,6), (11,14)

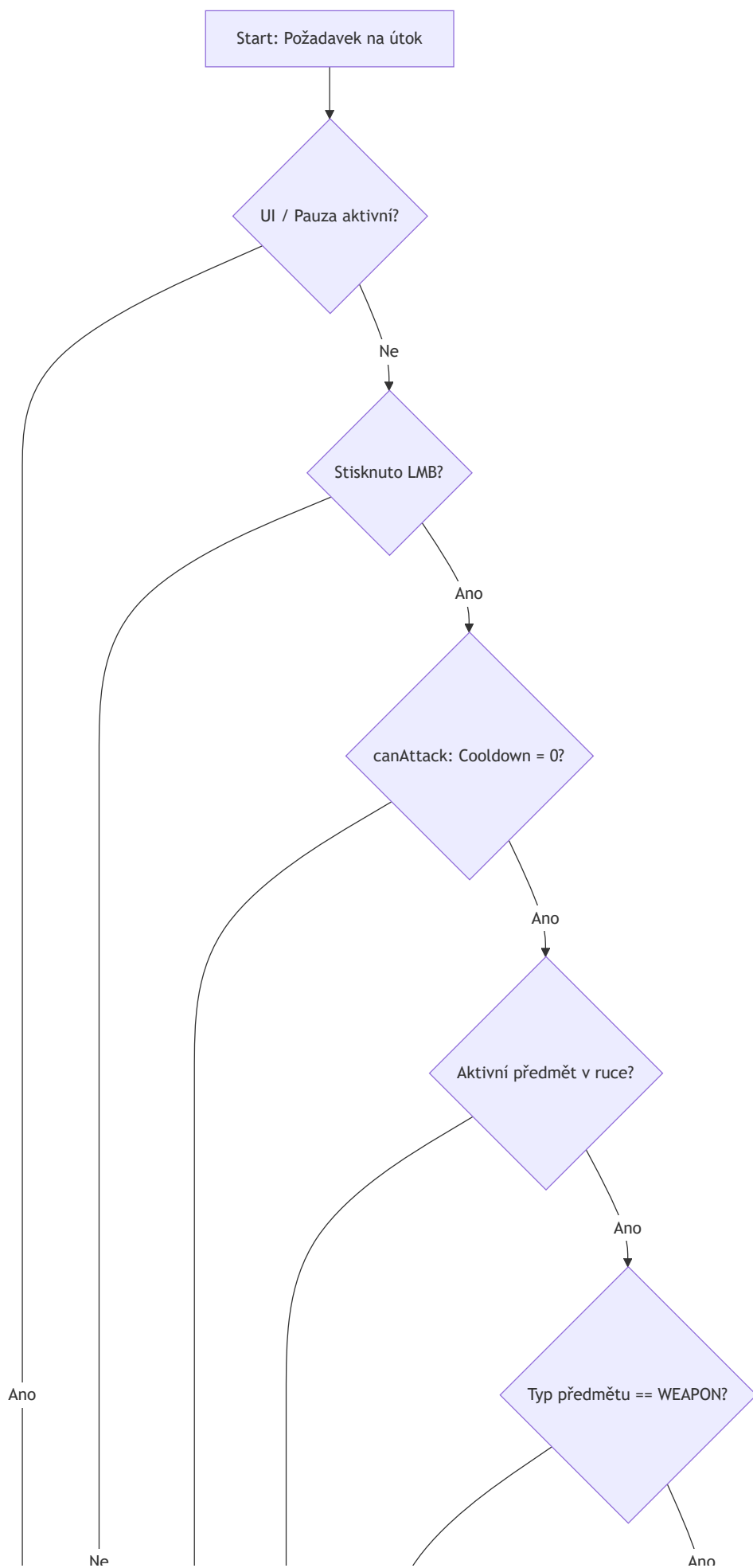
- **Klidový stav:** (2,7), (2,8), (2,9), (2,15), (6,7), (6,8), (6,9), (6,15), (7,7), (7,8), (7,9), (7,15), (12,7), (12,8), (12,9), (12,15)
- **Jeden krok:** (5,10), (5,11), (5,12), (5,13), (10,10), (10,11), (10,12), (10,13), (15,10), (15,11), (15,12), (15,13)

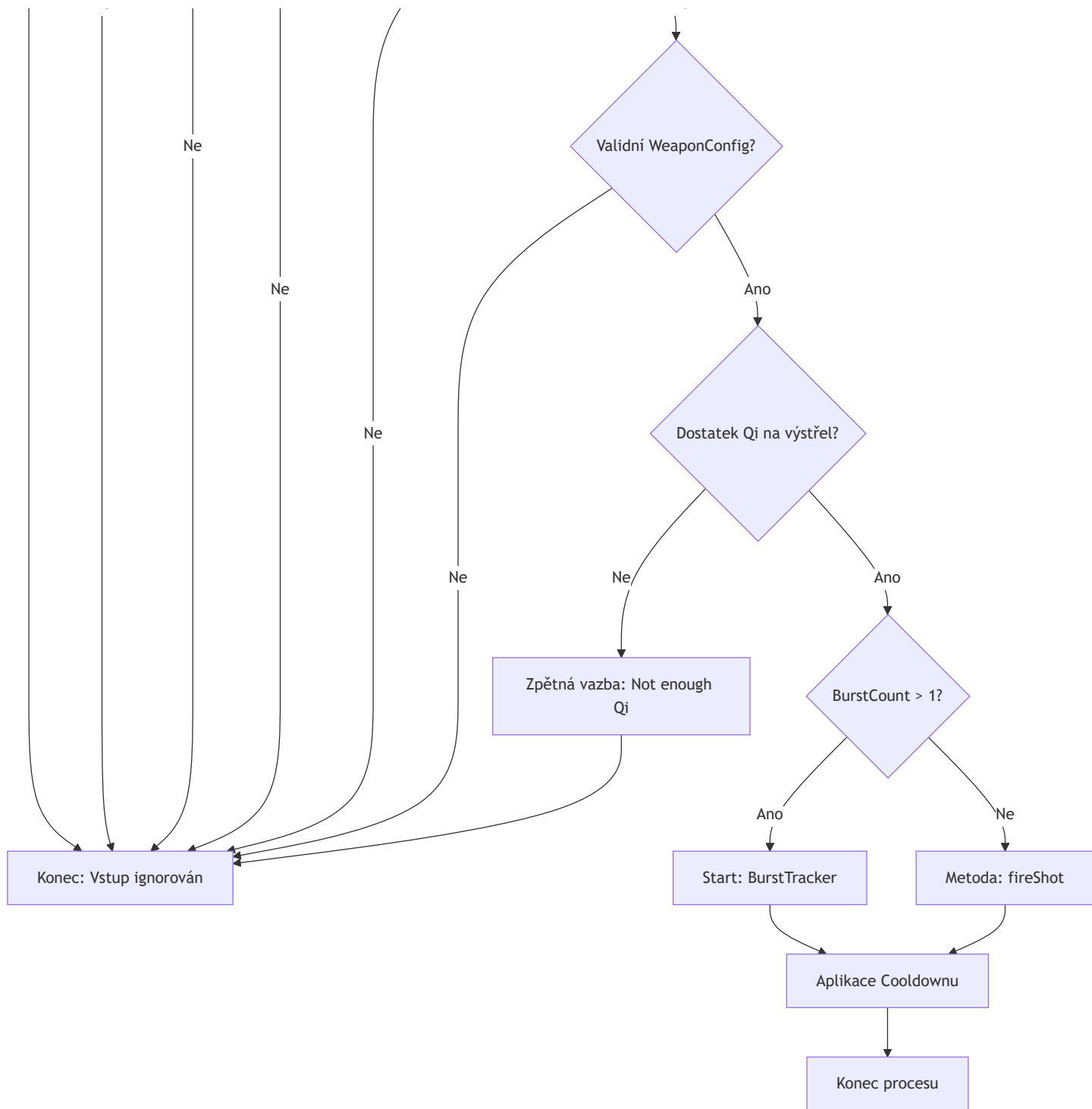
7.4.2 Testovací scénáře pro hloubku 2

- **P1:** 2 -> 9
- **P2:** 2 -> 7 -> 9
- **P3:** 1 -> 5 -> 11 -> 4 -> 6 -> 9
- **P4:** 2 -> 15 -> 10 -> 11 -> 5 -> 12 -> 9
- **P5:** 1 -> 14
- **P6:** 2 -> 8 -> 6 -> 8 -> 4 -> 14
- **P7:** 1 -> 6 -> 7 -> 7 -> 8 -> 5 -> 12 -> 8 -> 14
- **P8:** 1 -> 4 -> 6 -> 15 -> 12 -> 7 -> 15 -> 11 -> 14
- **P9:** 1 -> 5 -> 10 -> 13
- **P10:** 1 -> 5 -> 10 -> 10 -> 12 -> 15 -> 13
- **P11:** 1 -> 4 -> 4 -> 5 -> 11 -> 6 -> 8 -> 5 -> 13

7.5 Procesní diagram č. 2: Logika aktivace zbraně (Weapon Activation Flow)

Tento proces představuje komplexní rozhodovací logiku v třídě `CombatManager.handleFiring`, která orchestruje vstupy od uživatele, stav UI a RPG atributy postavy. Diagram obsahuje **8 rozhodovacích bodů**, což přesně odpovídá implementaci v kódu.







7.5.1 Analýza hran a TDL 2 (Hloubka 1 - Edge Coverage)

Pro ověření této logiky jsou definovány testovací sekvence pokrývající všechny větve implementace:

1. **UI blokace:** A -> D1(Ano) -> B

2. **Neúspěšný Qi Check:**

A -> D1(Ne) -> D2(Ano) -> D3(Ano) -> D4(Ano) -> D5(Ano) -> D6(Ano) -> D7(Ne) -> C -> B

3. **Úspěšný výstřel (Single):**

A -> D1(Ne) -> D2(Ano) -> D3(Ano) -> D4(Ano) -> D5(Ano) -> D6(Ano) -> D7(Ano) -> D8(Ne) -> F -> G -> H

4. **Úspěšná aktivace dávky (Burst):**

A -> D1(Ne) -> D2(Ano) -> D3(Ano) -> D4(Ano) -> D5(Ano) -> D6(Ano) -> D7(Ano) -> D8(Ano) -> E -> G -> H

8 Testování konzistence (Consistency Testing)

Pro testování konzistence dat v rámci životního cyklu entit používáme metodiku **CRUD matice**. Tato technika ověřuje, že data o entitách (hráč, nepřátelé, NPC) jsou v každém okamžiku v konzistentním stavu a systém na ně správně odkazuje v paměti i v seznamech subsystému `World`.

Testování se zaměřuje na potomky třídy `LivingEntity`, u kterých je správa životního cyklu (Spawn -> Update -> Despawn) nejkritičtější.

8.1 CRUD matice

Následující tabulka definuje operace (Create, Read, Update, Delete) a přiřazuje jim identifikační čísla pro následné testování sekvencí průchodů.

Operace	Typ (CRUD)	ID	Popis v kontextu DaoEngine
Vytvoření	C	1	Spawn entity skrze <code>EnemyRegistry</code> nebo <code>LevelLoader</code> .
Vykreslení	R	2	Volání metody <code>render()</code> – vizuální potvrzení existence objektu.
Pohyb	U	3	Aktualizace souřadnic <code>x</code> , <code>y</code> v rámci metody <code>update()</code> .
Výpis seznamu	R	4	Kontrola přítomnosti v <code>Level.getEntities()</code> listu.
Opuštění mapy	U	5	Despawn / Deaktivace entity (např. při vzdálení se hráče).
Příchod	U	6	Znovunačtení / Aktivace entity při návratu do oblasti.
Změna statistik	U	7	Aplikace damage nebo efektů v rámci <code>AttributeSet</code> .
Smrt / Odstranění	D	8	Úplné smazání instance z paměti a všech seznamů entit.

8.2 Testovací sekvence (Průchody konzistence)

Testování probíhá formou sekvenčních volání, kde čísla v závorkách `[R]` značí kontrolní čtení stavu pro ověření, že data nebyla poškozena předchozí operací.

- **Standardní životní cyklus:** 1 - 2 - `[4]` - 3 - 4 - `[2]` - 5 - 4 - `[2]` - 6 - 2 - `[4]` - 7 - 4 - `[2]`
 - *Analýza:* Entita je vytvořena, vykreslena, probíhá pohyb, dočasně opouští mapu (despawn), vrací se (respawn) a na závěr jsou upraveny její statistiky. V každém kroku `[4]` je ověřena integrita listu entit.
- **Destrukční cyklus (Smrt):** 1 - 2 - `[4]` - 3 - 4 - `[2]` - 5 - 4 - `[2]` - 6 - 2 - `[4]` - 8 - 4 - `[2]`
 - *Analýza:* Podobný průběh jako výše, ale sekvence končí operací 8 (definitivní odstranění). Následná kontrola `[4]` musí potvrdit, že v seznamu již entita nefiguruje, čímž je potvrzena konzistence uvolňování paměti.

9 Testovací scénáře

V této sekci je popsán komplexní integrační scénář, který simuluje standardní průchod hráče od spuštění aplikace až po úspěšné uložení postupu. Tento scénář slouží k ověření funkční provázanosti všech hlavních subsystémů DaoEngine.

9.1 Scénář č. 1: Průchod komplexní herní session (Quest & Persistence)

Název: Integrace herních mechanik, dialogů a perzistence.

Shrnutí: Spuštění hry, interakce s NPC, přijetí úkolu, souboj, splnění úkolu a uložení postupu do souboru.

Pracnost: Střední až Vysoká (pokrývá všechny hlavní subsystémy).

Vstupní podmínky:

- Aplikace je vypnuta, složka se savy je prázdná.
- Konfigurace `game_config.json` je v defaultním stavu.

Testovací data:

- **Postava:** Player (100 HP, 100 Qi, Base Speed 2.5).
- **NPC:** `gatekeeper_tutorial` (souřadnice definované v `map1.json`).
- **Nepřítel:** `spirit_beast` (Level 1).
- **Dovednost:** `fiery_palm` (Technika typu Fire).

Popis kroků (Test Procedure):

1. **Inicializace:** Uživatel spustí hru a v menu zvolí "New Game".
2. **Průzkum:** Pomocí `WASD` se hráč přesune k nejbližšímu NPC (Tutorial NPC).
3. **Interakce:** Stisknutím klávesy `E` vyvolá dialogové okno.
4. **Progrese:** Hráč v dialogu potvrdí přijetí úkolu na eliminaci nepřítele.
5. **Souboj:** Hráč vyhledá v herním světě entitu `spirit_beast` a zničí ji útokem (Lecké kliknutí).
6. **Regenerace:** Po souboji hráč využije klávesu `SPACE` pro meditaci na Spirit Veinu, aby obnovil spotřebovanou energii Qi.
7. **Dokončení:** Hráč se vrátí k NPC a opětovným stisknutím `E` odevzdá hotový úkol.
8. **Uložení:** Hráč stiskne `ESC`, v menu vybere slot 1 a potvrdí "Save".

Očekávaný výsledek:

- **Kroky 1-3:** Aplikace se spustí bez výjimek v logu. `PlayState` se správně inicializuje a vykreslí mapu. Interakční symbol se u NPC objeví při přiblížení na vzdálenost $< 150\text{px}$.
- **Kroky 4-5:** `QuestManager` zaregistruje aktivní úkol. Při souboji `CombatManager` správně aplikuje damage a `AttributeSet` odečte HP cíli. `ParticleManager` generuje vizuální particle efekty při každém zásahu.
- **Kroky 6-7:** `EventManager` po smrti entity doručí signál `QuestManageru`, který aktualizuje stav úkolu v HUDu (1/1). NPC v dialogu změní svůj text a hráč obdrží odměnu (např. Cultivation XP).
- **Krok 8:** Systém vyvolá asynchronní vlákno `SaveTask`. Na disku vznikne validní JSON soubor `save_slot_1.json`, který po manuální kontrole obsahuje: `hp: 100`, `completedQuests: ["tutorial_01"]` a správnou mapovou pozici.

9.2 Scénář č. 2: Smrt a znovuzrození (Death & Respawn Flow)

Název: Cyklus selhání, Game Over a obnova stavu.

Shrnutí: Hráč klesne na 0 HP, dojde k přepnutí do `GameOverState` a následnému načtení poslední pozice.

Pracnost: Střední.

Vstupní podmínky:

- Hráč je v `PlayState` uprostřed souboje.
- Existuje alespoň jedno předchozí uložení (`save_slot_1.json`).

Testovací data:

- **Postava:** HP = 5, Qi = 0.
- **Nepřítel:** `boss_entity` s vysokým poškozením (20+).

Popis kroků (Test Procedure):

1. **Poškození:** Hráč záměrně inkasuje zásah od nepřítele, který sníží jeho HP na 0.
2. **Stav:** Sleduje se přechod z `PlayState` do `GameOverState` .
3. **UI:** Na obrazovce se zobrazí nápis "You have perished" a tlačítko "Load Last Save".
4. **Obnova:** Hráč klikne na tlačítko načtení.
5. **Verifikace:** Sleduje se přechod do `LoadingState` a následný návrat do `PlayState` .

Očekávaný výsledek:

- **Krok 1-2:** Při detekci `HP <= 0` v `AttributeSet` vyvolá engine událost, která zastaví herní smyčku a přepne `GameState` .
- **Krok 3:** UI správně vykreslí overlay přes canvas. Hudba se změní na "death_theme".
- **Krok 4-5:** `SaveManager` načte data ze slotu. Hráč se objeví na poslední uložené pozici s plným zdravím (dle saveu). Veškeré entity v levelu jsou resetovány do výchozího stavu dle `LevelLoaderu` .

YOU DIED

Your soul returns to the cycle of reincarnation...

Try Again

Main Menu

10 Seznam nalezených chyb

Během procesu testování (zejména při unit testech a integračních testech soubojového systému) byly identifikovány a následně opraveny následující chyby, které by bez metodického testování pravděpodobně zůstaly neodhaleny:

- **Logická chyba v regeneraci Qi:** V třídě `Player` byla nalezena nelogická podmínka `(qi < maxQi && !isMeditating)`, která v některých okrajových případech (při současném zásahu nepřítelem) blokovala pasivní regeneraci energie na Spirit Veinech. Opraveno na korektní kontrolu stavů.
- **Duplikace eventů v `questManageru`:** Při specifické kombinaci rychlého útoku a zničení entity mohl `EventManager` odeslat duplicitní signál o smrti. To vedlo k nekorektnímu stavu questu (např. 2/1 splněných cílů). Vyřešeno přidáním validace unikátních ID událostí.
- **Race condition při ukládání:** Při manuálním testování bylo zjištěno, že pokud hráč vypne hru okamžitě po kliknutí na uložení, asynchronní `SaveTask` nestihne dokončit zápis a JSON soubor zůstane poškozen. Opraveno implementací synchronizačního mechanismu v rámci `onCloseRequest` okna aplikace.

11 Závěr

Semestrální práce pro mě byla unikátní zkušeností především kvůli možnosti otestovat svůj vlastní projekt DaoEngine. Jako autor aplikace jsem se musel vypořádat s největším úskalím testera s profesní slepotou. Proces návrhu testovací sady mě donutil přepnout z módu „vývojáře“, který kód intuitivně používá tak, aby fungoval, do módu „analytika“, který aktivně hledá cesty, jak systém rozbít v jeho hraničních stavech.

Metodický přístup k verifikaci mi ukázal, že i v projektu, který jsem považoval za relativně stabilní, se pod povrchem skrývaly logické nedostatky. Díky aplikaci technik jako MC/DC, BVA nebo Pairwise testing se mi podařilo identifikovat a následně opravit kritické chyby, zejména v asynchronním ukládání dat (race conditions) a v matematickém modelu regenerace atributů. Právě nalezení těchto reálných bugů v mém vlastním kódu pro mě bylo nejlepším důkazem, že testování není jen teoretická nadstavba, ale nezbytná součást inženýrské práce.

Tato práce mi pomohla DaoEngine stabilizovat a vytvořit z něj robustnější aplikaci. Získané znalosti o procesním testování a analýze rizik vnímám jako zásadní posun ve své programátorské praxi. Pochopení rozdílu mezi „fungujícím kódem“ a „ověřeným softwarem“ je pro mě hlavním přínosem celého předmětu a cenným základem pro mé budoucí projekty i profesní působení v softwarovém inženýrství.